Enabling Safe Usage of Linux in Systems: Progress to Date

Texas Linux Fest
Nov 7th, 10:00-10:50 CST

Kate Stewart
kstewart@linuxfoundation.org



ELISA
Enabling Linux in Safety Applications

Aerospace · Automotive · Linux Features
OS Engineering Process · Railways · Safety Architecture
Systems · Tools Lighthouse · Space Grade Linux



source: https://www.linuxfoundation.org/hubfs/Research%20Reports/LFResearch_35YearsofLinux_Report_2026.pdf

License: CC-BY-4.0

Where is Linux Being Used Today?

100% Supercomputer Market

82% Smartphone Market Share

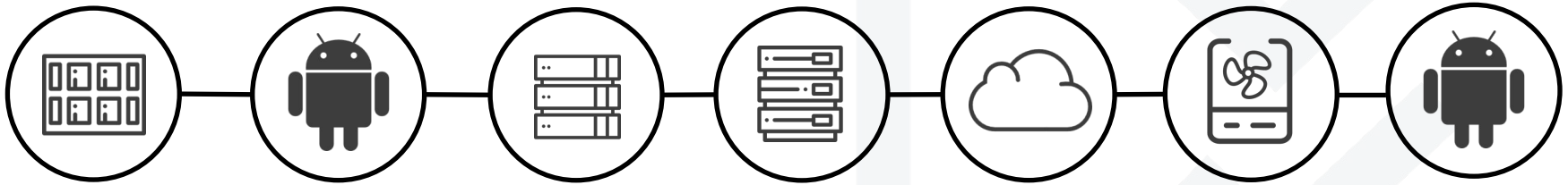
2nd To Windows in Enterprise

90% Mainframe Customers

90% Public Cloud Workload

69% Embedded Systems Market

#1 Internet Client



Every market Linux has entered it eventually dominated

ELISA
Enabling Linux in Safety Applications

License: CC-BY-4.0

Where is Embedded Linux Being Used Today?

Today, Linux powers 4B Android devices, 30M Chromebooks, 200M+ servers, all of the top 500 supercomputers, and Apple's 5G modems.



Autonomous space missions, satellite constellations, and commercial flight drones use Linux to execute real-time navigation algorithms and complex telemetry processing.



In healthcare, embedded Linux powers medical equipment, diagnostic tools, and genomic research while adhering to international safety certification standards.



By enabling accessible digital payments, agricultural tools, clean energy, and global connectivity, the Linux kernel silently powers essential daily experiences for millions worldwide.





Automotive Grade Linux unifies the vehicle software stack under a collaborative framework with long-term support for safety-critical software.

Thanks to real-time kernel capabilities and hardware portability, Linux dominates local edge devices, regional data centers, and 5G network towers.



ELISA
Enabling Linux in Safety Applications

source: https://www.linuxfoundation.org/hubfs/Research%20Reports/LFResearch_35YearsofLinux_Report_2026.pdf

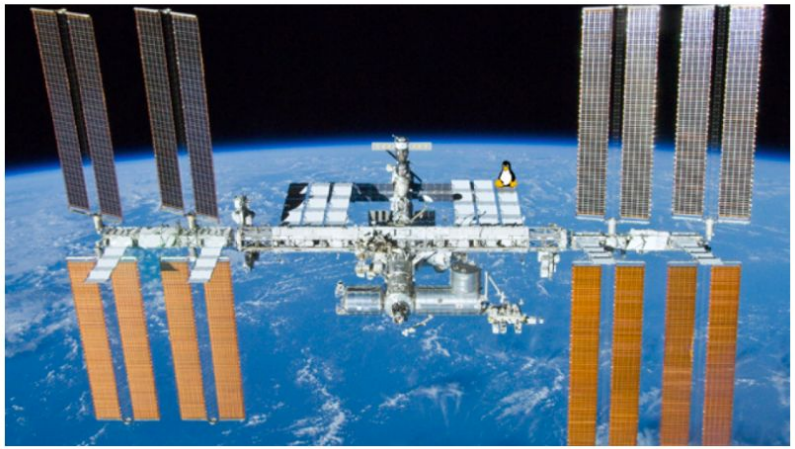
License: CC-BY-4.0

Home > Extreme

International Space Station switches from Windows to Linux, for improved reliability

The United Space Alliance, which manages the computers aboard the International Space Station in association with NASA, has announced that the Windows XP computers aboard the ISS have been switched to Linux. "We migrated key functions from Windows to Linux because we needed an operating system that was stable and reliable."

By Sebastian Anthony May 9, 2013



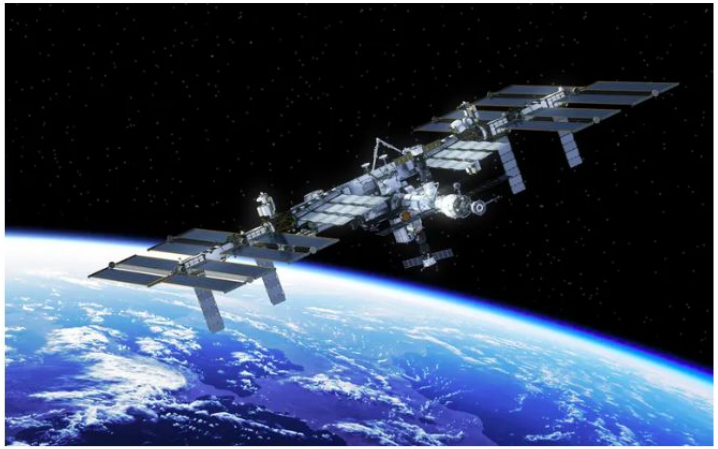
Source:https://www.extremetech.com/extreme/155392-international-space-station-switches-from-windows-to-linux-for-improved-reliability

The ISS just got its own Linux supercomputer

The International Space Station will host a test of computer hardware that's vital for any future missions to Mars.



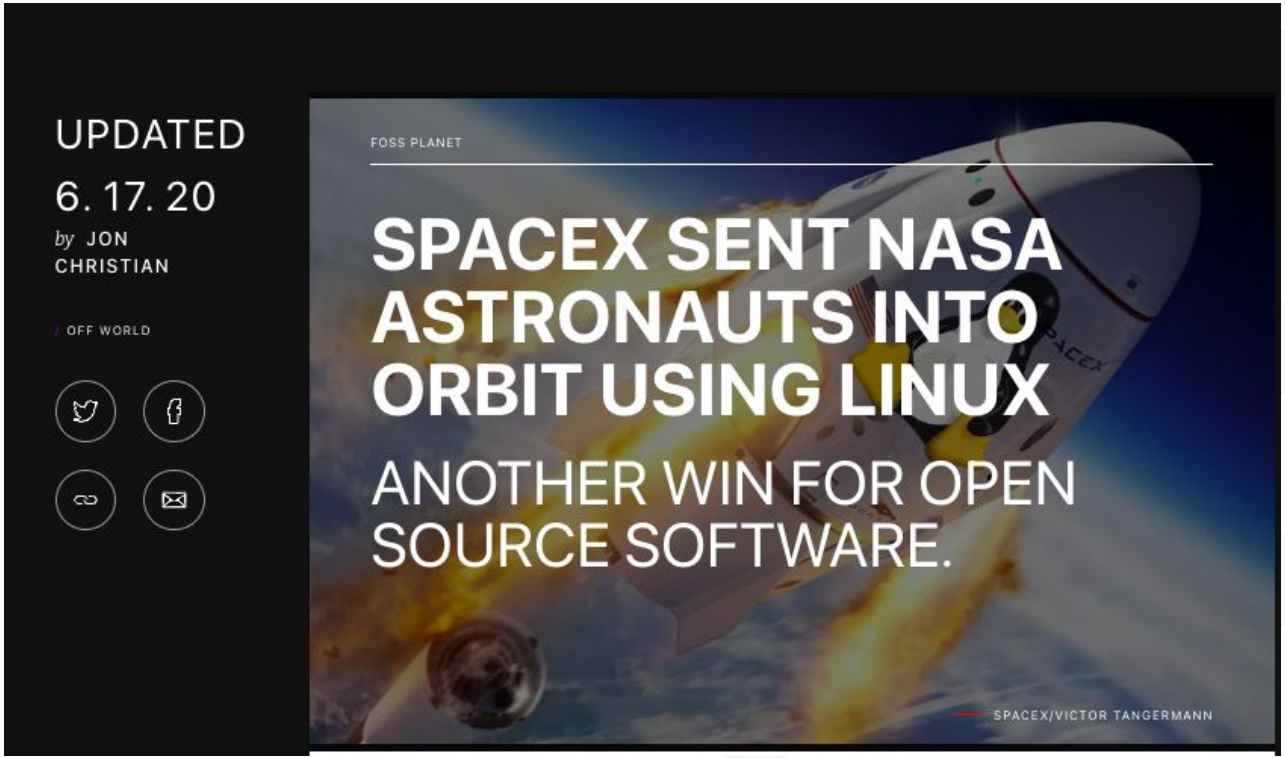
Written by **Steve Ranger,** Global News Director on Sept. 21, 2017



Source:https://www.extremetech.com/extreme/155392-international-space-station-switches-from-windows-to-linux-for-improved-reliability

ELISA
Enabling Linux in Safety Applications

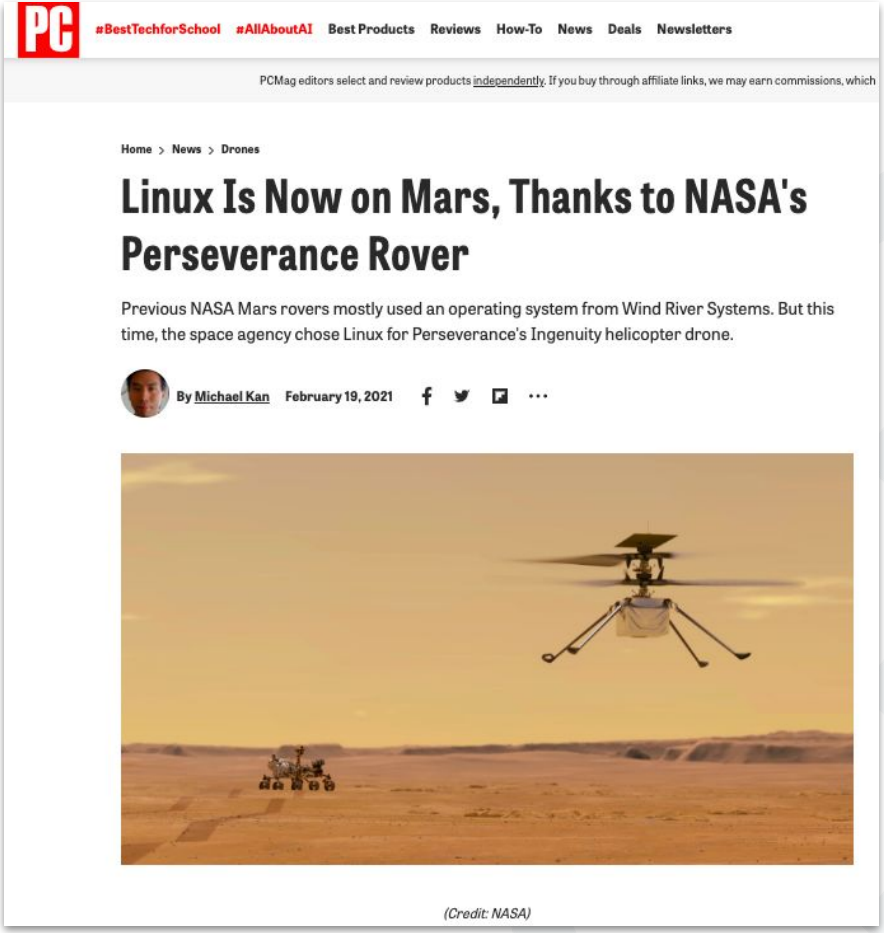
License: CC-BY-4.0



Source: https://futurism.com/the-byte/spacex-nasa-astronauts-linux

ELISA
Enabling Linux in Safety Applications

License: CC-BY-4.0



PC #BestTechforSchool #AllAboutAI Best Products Reviews How-To News Deals Newsletters

PCMag editors select and review products independently. If you buy through affiliate links, we may earn commissions, which

Home > News > Drones

Linux Is Now on Mars, Thanks to NASA's Perseverance Rover

Previous NASA Mars rovers mostly used an operating system from Wind River Systems. But this time, the space agency chose Linux for Perseverance's Ingenuity helicopter drone.

By Michael Kan February 19, 2021

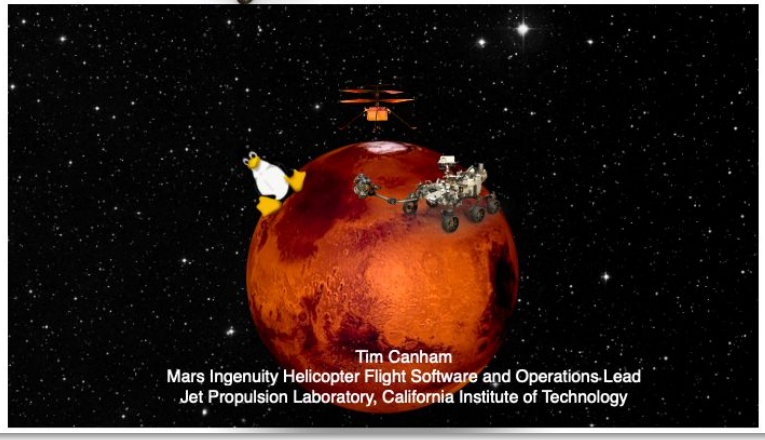
(Credit: NASA)

Source: https://www.pcmag.com/news/linux-is-now-on-mars-thanks-to-nasas-perseverance-rover

ELISA
Enabling Linux in Safety Applications

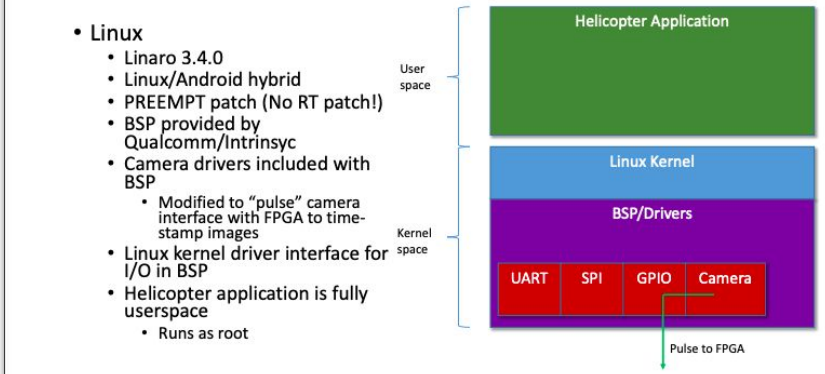
License: CC-BY-4.0

Linux on Mars



Mars Helicopter Operating System and Software

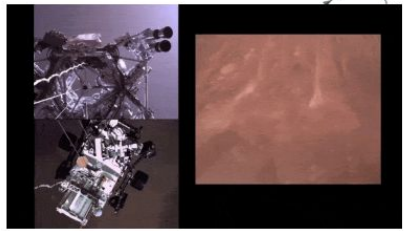
- Linux
  - Linaro 3.4.0
  - Linux/Android hybrid
  - PREEMPT patch (No RT patch!)
  - BSP provided by Qualcomm/Intrinsyc
  - Camera drivers included with BSP
    - Modified to "pulse" camera interface with FPGA to time-stamp images
  - Linux kernel driver interface for I/O in BSP
  - Helicopter application is fully userspace
    - Runs as root



© 2021 California Institute of Technology. Government sponsorship acknowledged.

Perseverance Rover EDL Cameras

- Perseverance Rover also had Linux-based landing camera system
  - Not involved in guidance, just recorded landing
- Ruggedized Intel Atom PC
  - More like a conventional PC
- USB cameras
  - USB cabling with hubs throughout vehicle
  - FTDI to rover interface UART
- Linux x86 kernel 4.15.7
- Used much open-source including ffmpeg and Python



Conclusions

- Linux boosted our ability to develop quickly
  - We had standard I/O drivers
  - Manufacturer BSP was available
  - Shell/adb interface made testing much easier
  - COTS facilities like Wi-Fi, USB and standard I/O made test support equipment *much* easier
  - Allowed early prototyping on other platforms like Raspberry Pi
- Linux did very well, as long as you were aware of its limitations
  - Not real time, so built in robustness to slips
    - RT patch probably would have been better, but not available on our kernel
  - Avoid file I/O during performance critical times
  - Build in file-system level protections (ex. read-only partitions for software/Linux executables)
- Future of Linux in space exploration is rosy!

License: CC-BY-4.0

Source: Embedded Linux Conference 2021 https://elinux.org/images/5/5a/1._TIMOTHY_CANHAM.pdf



BEYOND GLOBAL DOMINANCE: OPEN SOURCE IN ORBIT AND BEYOND

David VomLehn
2023 June 28

Embedded Linux Conference 2023
Video:https://www.youtube.com/watch?v=I_LbbvA0NiU&list=PLbzoR-pLrL6qMzIhKVe9lMFnYvFZqrp03&index=5



Controls and Infotainment System Change over 30 Years!



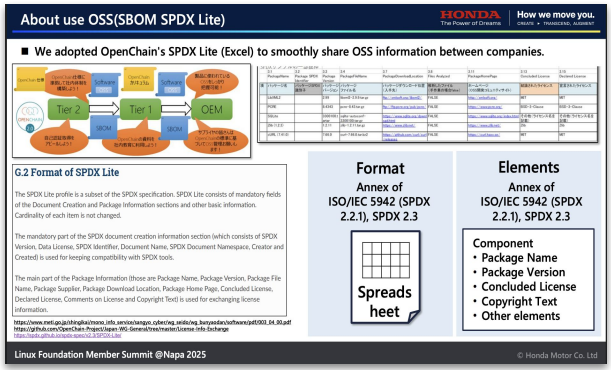


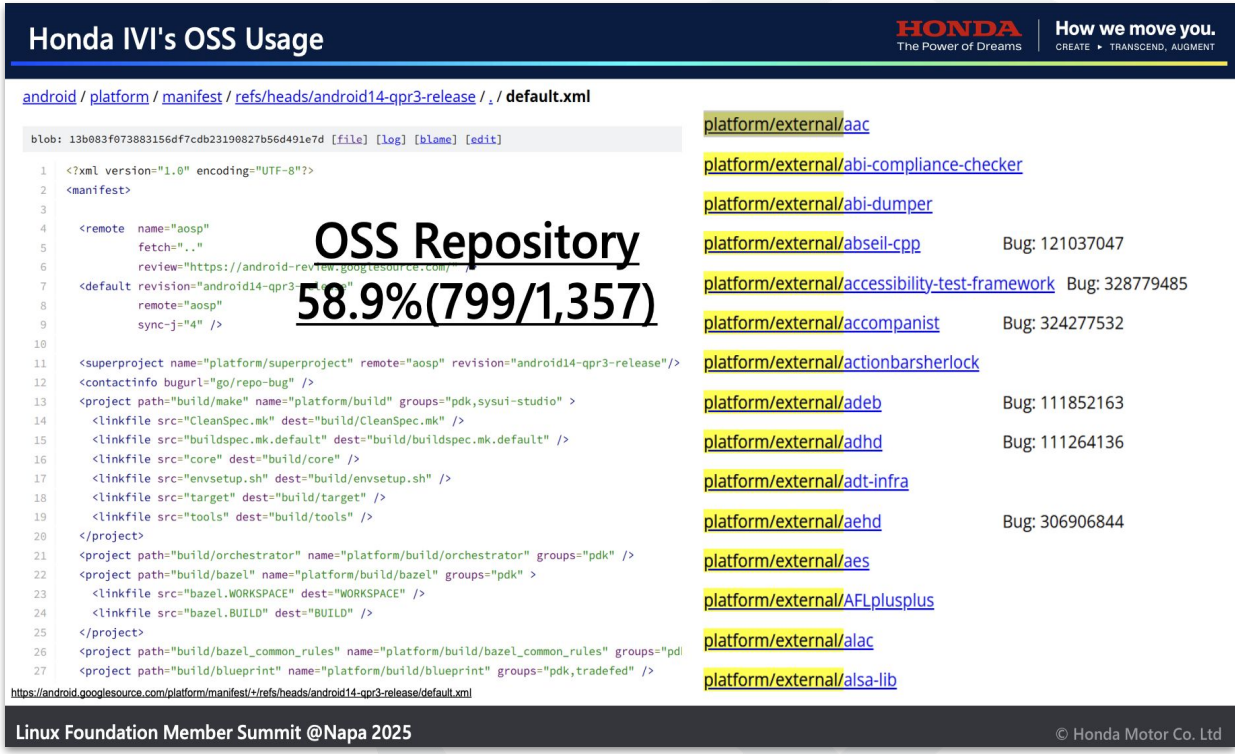
ELISA
Enabling Linux in Safety Applications

License: CC-BY-4.0

Supply Chain behind a modern car …



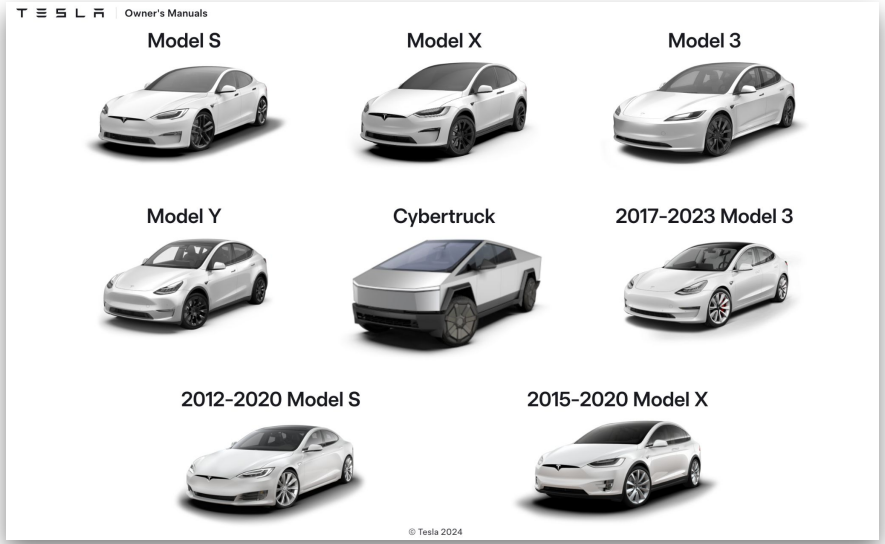




Source: https://static.sched.com/hosted_files/lfms25/e0/250312_UnderstandingandManagingSBOMsinModernAutomotiveVehicles_LFMemberSummit2025.r2.pdf

License: CC-BY-4.0

Linux is being used in Safety Critical Systems today…



source: https://www.tesla.com/ownersmanual



source: https://www.spacex.com/mission/

ELISA
Enabling Linux in Safety Applications

Work in Progress - License: CC-BY-4.0

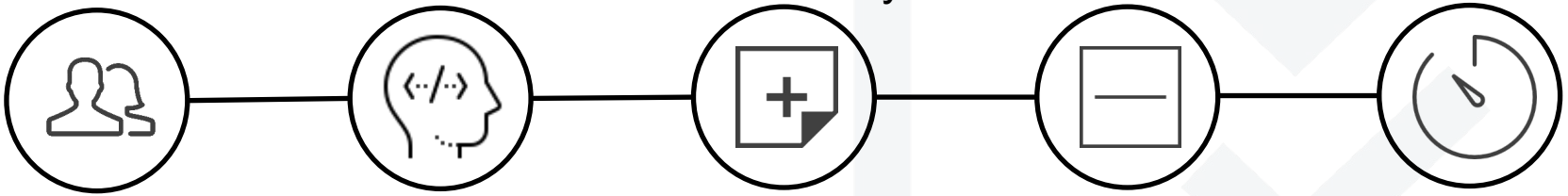
Linux Rate of Change

7.3 Linux Kernel Statistics*

3,021
Contributors From
232 Organizations

14,603
Lines of Code
Added Daily

12.4
Changes Per
Hour



807
New
Contributors

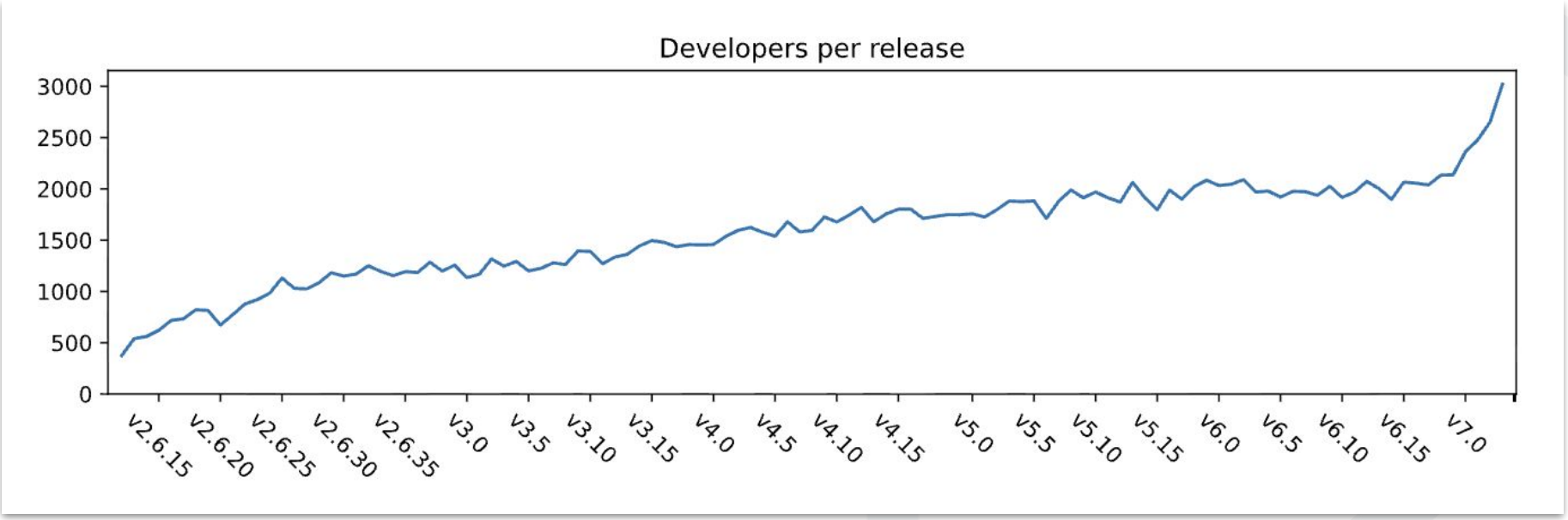
4,285
Lines of Code
Removed Daily

Time period for 7.3: 2026/8/16-2026/10/18=63 days
* Source: https://lwn.net/Articles/1100241/

License: CC-BY-4.0

ELISA
Enabling Linux in
Safety Applications

Kernel Contributions Accelerating with AI

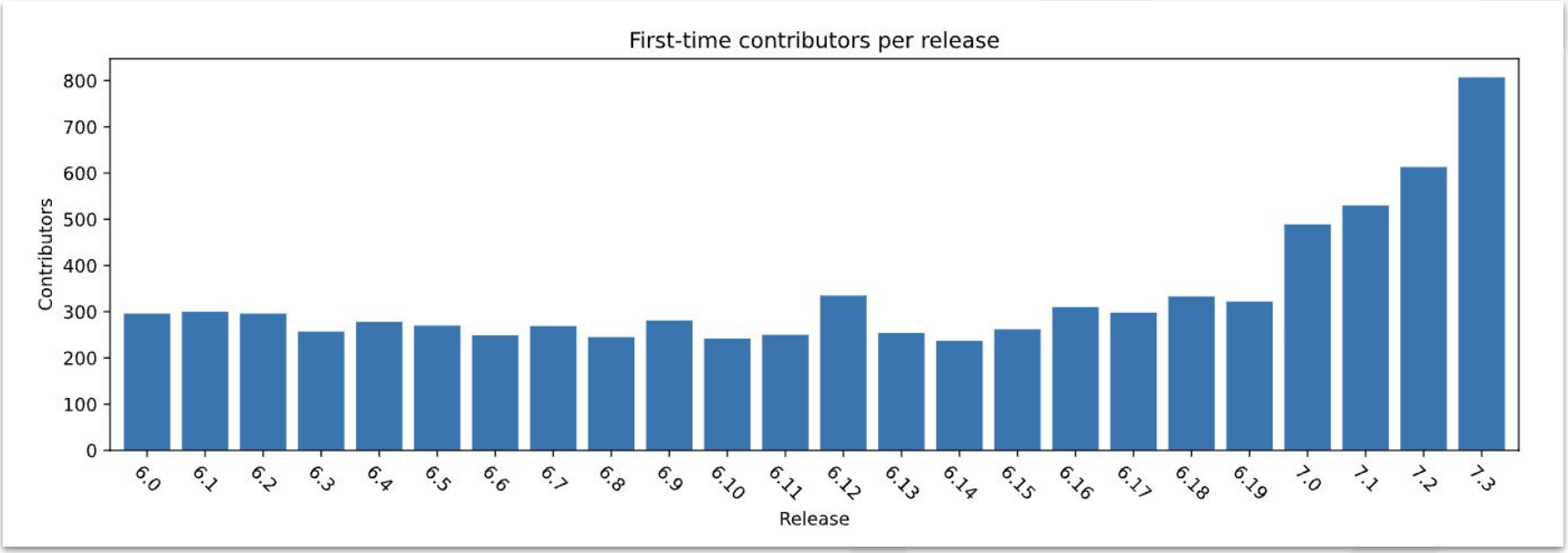


Time period for 7.3: 2026/8/16-2026/10/18=63 days
* Source: https://lwn.net/Articles/1100241/

ELISA
Enabling Linux in Safety Applications

License: CC-BY-4.0

More New Kernel Contributions with AI assist



Time period for 7.3: 2026/8/16-2026/10/18=63 days
* Source: https://lwn.net/Articles/1100241/

ELISA
Enabling Linux in Safety Applications

License: CC-BY-4.0

Safety Standards have emerged from 70+ years of analysis to mitigate systemic risk.

License: CC-BY-4.0

What is Functional Safety?

Definition of Safety

The freedom from unacceptable risk of physical injury or of damage to the health of people, either directly, or indirectly because of damage to property or the environment.

Definition of Functional Safety

The part of safety that depends on a system or equipment operating correctly in response to its inputs. Detecting potentially dangerous conditions, resulting either in the activation of a protective or corrective device or mechanism to prevent hazardous events or in providing mitigation measures to reduce the consequences of the hazardous event.

License: CC-BY-4.0

In Functional Safety you expect…

…that the software:

- does behave as specified,
- does not interfere or impair other system components
- all possible erroneous events are addressed somehow or somewhere…

and you have sufficient evidence to prove this.

License: CC-BY-4.0



Chemical



Financial



Commercial Facilities



Food & Agriculture



Communi-cations



Government Facilities



Critical Manu-facuring



Healthcare & Public Care



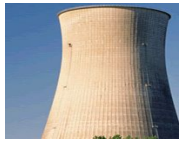
Dams



Information Technology



Defense Industrial Base



Nuclear Reactors, Materials, & Waste



Emergency Services



Transpor-tation Systems



Energy



Water & Wastewater Systems

Source: https://www.cisa.gov/critical-infrastructure-sectors

June 2022: Japan Cybersecurity & Critical Infrastructure

Critical Infrastructure

Since 2005, the 'Cybersecurity Policy for Critical Infrastructure Protection' has been set as a common action plan shared between the government, which bears responsibility for promoting independent measures by CI operators relating to CI cybersecurity and implementing other necessary measures, and CI operators which independently carry out relevant protective measures, and the new edition was published in 2022.
This document identifies the 14 sectors as critical infrastructure and it expects stakeholders to undertake the five measures as below.

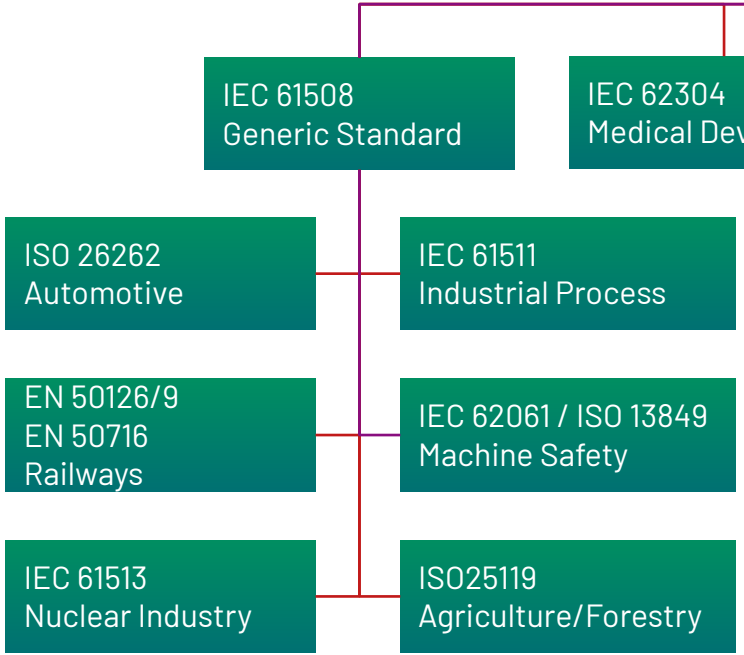
1. Enhancement of Incident Response Capability
2. Maintenance and Promotion of the Safety Principles
3. Enhancement of Information Sharing System
4. Utilization of Risk Management
5. Enhancement of the Basis for CIP

2. Maintenance and promotion of the safety principles	Basically keep the element of "[1] Maintenance and promotion of the safety principles"	○ Clarify that safety standards, etc., that contribute to the enhancement of incident response capability and risk management are to be developed. ○ Consider survey methods capable of continuously improving the activities of CI operators.

The Cybersecurity Policy for Critical Infrastructure Protection
Full Text
Guideline for Establishing Safety Principles for Ensuring Information Security of Critical Infrastructure(5th Edition)(Revised on May 2019)
Risk Assessment Guide Based on the Concept of Mission Assurance in Critical Infrastructure (1st Edition)(Revised on May 2019)

source: https://www.nisc.go.jp/eng/index.html#sec4

Samples of safety (integrity) standards



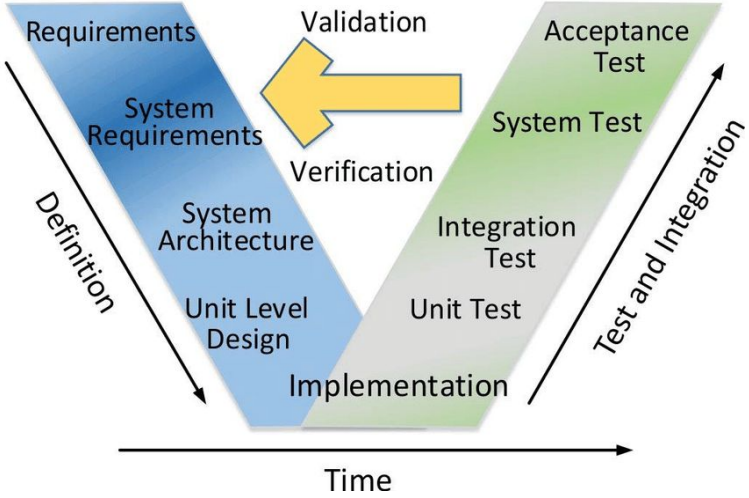
- Standards represent industry best practice and recommend to use these practices
- Share similar needs… requirements, documentation, testing
- Rigor on measures/demands may differ

All system parts need to be known, tested and managed

Work in Progress - License: CC-BY-4.0

Safety Engineering 101: the "V-Model"

- The V-model for functional safety development originated from systems engineering practices to structure the process of designing and validating complex systems.
- It was later adapted and widely adopted in the automotive industry and other safety-critical sectors as a **framework for ensuring the systematic integration of safety requirements and processes throughout each stage of the development lifecycle.**
  - **ISO 26262** in the automotive industry
  - **IEC 61508** for industrial systems
  - **DO-178C** in aerospace



Source Image image provided under CC-4.0
https://www.researchgate.net/figure/The-functional-safety-development-via-V-model-14_fig4_362572593

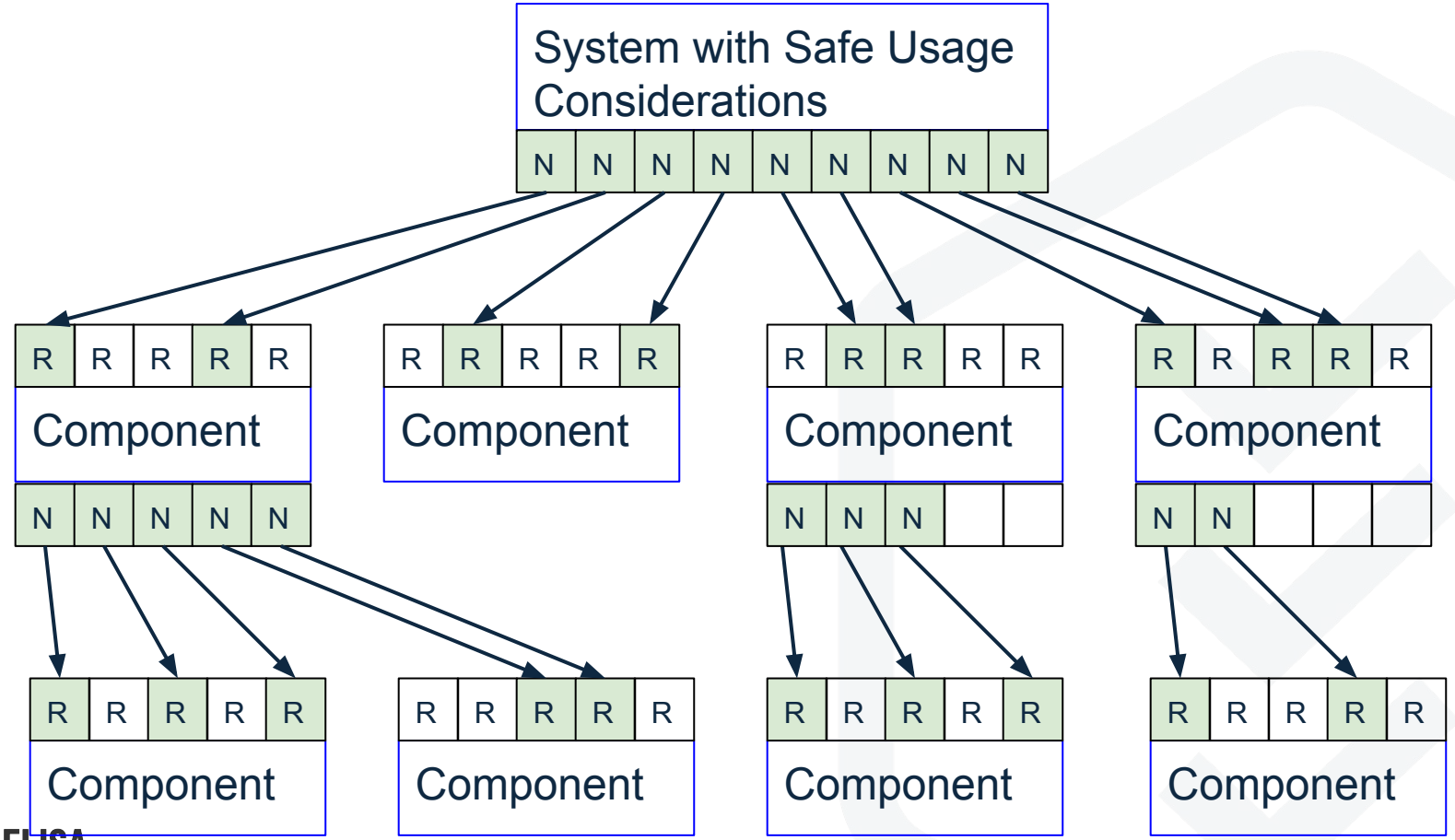
Approximate cross-domain mapping of ASIL

Domain	Domain-Specific Safety Levels					
Automotive (ISO 26262)	QM	ASIL A	ASIL B	ASIL C	ASIL D	-
General (IEC 61508)	-	SIL-1	SIL-2		SIL-3	SIL-4
Railway (CENELEC 50126/128/129)	-	SIL-1	SIL-2		SIL-3	SIL-4
Space (ECSS-Q-ST-80)	Category E	Category D	Category C		Category B	Category A
Aviation: airborne (ED-12/DO-178/DO-254)	DAL-E	DAL-D	DAL-C		DAL-B	DAL-A
Aviation: ground (ED-109/DO-278)	AL6	AL5	AL4	AL3	AL2	AL1
Medical (IEC 62304)	Class A	Class B			Class C	-
Household (IEC 60730)	Class A	Class B			Class C	-
Machinery (ISO 13849)	PL a	PL b	PL c	PL d	PL e	-

Source: https://en.wikipedia.org/wiki/Automotive_Safety_Integrity_Level#Comparison_with_Other_Hazard_Level_Standards

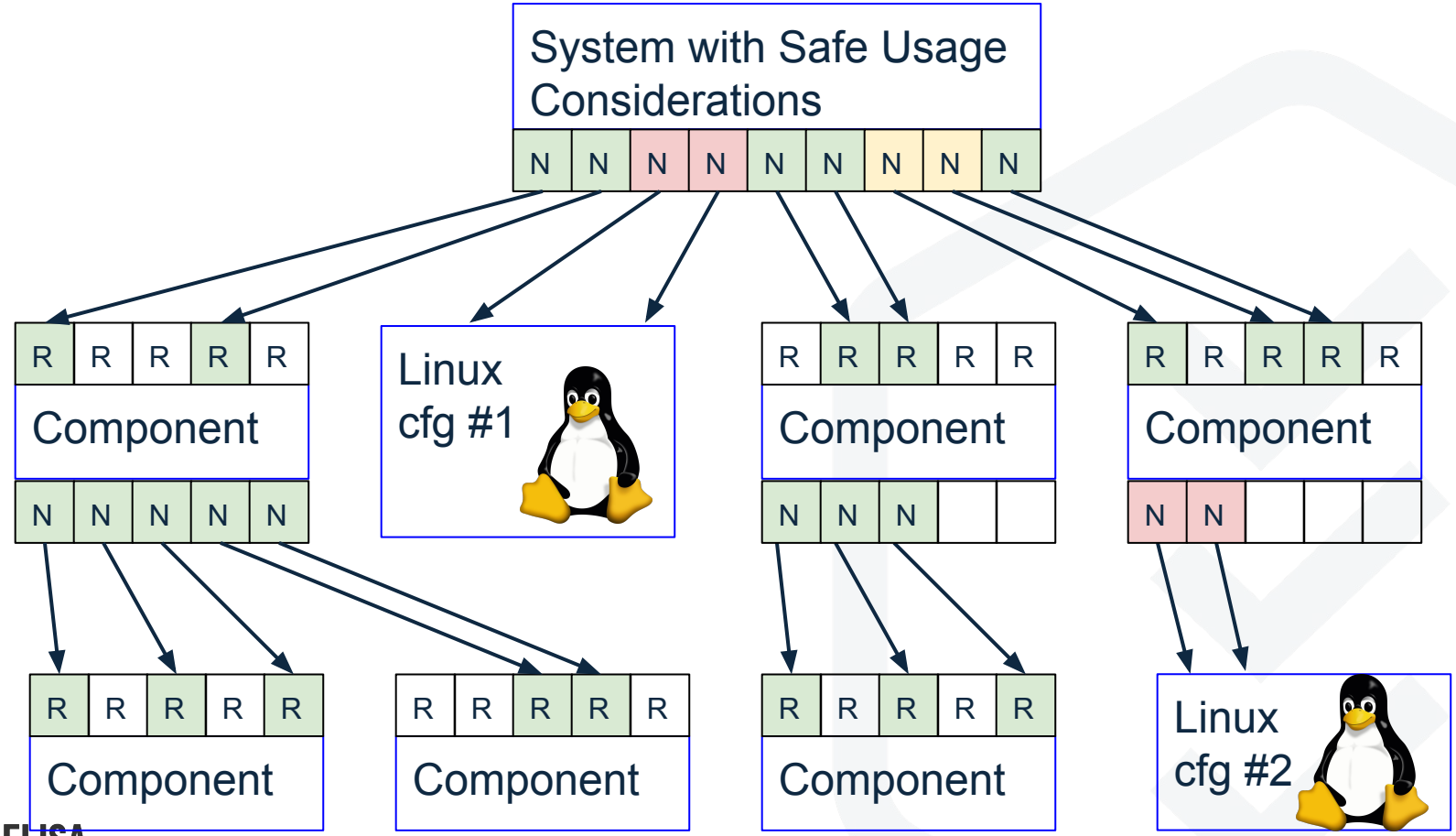
ELISA
Enabling Linux in Safety Applications

License: CC-BY-4.0



ELISA
Enabling Linux in Safety Applications

Work in Progress - License: CC-BY-4.0



ELISA
Enabling Linux in Safety Applications

Work in Progress - License: CC-BY-4.0

How can we address the "GAPS" to enable safe usage of systems including Linux and keep up with the rate of change of Linux?

Work in Progress - License: CC-BY-4.0

Challenge with Safe Usage of Linux Kernel

Each patch has a reason for being added to tree - "what" & "why"

- Frequently contained in patch series overview, but may be part of email discussion.
- Understanding "what" the code should do, is considered as a "**requirement**" on a component (like the kernel) when doing functional safety system analysis.
- Testing the functionality for when it **works**, and when it **does not work** is needed as "evidence" that is required to assess "Safe Usage".

Challenge: The Linux Kernel has no way of systematically capturing "what" the code is expected to do in a machine readable form.

If the "assertions about the code" (may be referred to as specifications or requirements) are reverse engineered by others, where should they be stored, so they can be reviewed by maintainers and other experts?

What mechanisms should be used to link the code & tests to these requirements?

Safety Standards Need Automation



- Safety Standards expect to know
  - The **source** code at the time of production release
  - The **documentation of use** associated with the code
  - The **configuration** used to **build** the production software
  - The **specific versions of the tools** used to build the software
  - The **specific hardware** that the software is running on
- Safety Standards Configuration Management (CM) Requirements are greatly simplified by leveraging software bill of materials (SBOM) transparency.
  - An SBOM supports capturing the details of what is in a specific release and supports determining what went wrong if a failure occurs.
  - The goal is to be able to **rebuild exactly** what the executable or binary was at the time of release.
- To learn more, see: https://www.linux.com/featured/sboms-supporting-safety-critical-software/

Work in Progress - License: CC-BY-4.0

Maintenance and Promotion of Safety Principles

Safety Standards are looking for:

- **Unique ID**, something to uniquely identify the version of the software you are using.
  - Variations in releases make it important to be able to distinguish the exact version you are using.
  - The unique ID could be as simple as using the hash from a configuration management tool, so that you know whether it has changed.
- **Dependencies of the component**
  - Any chained dependencies that a component may require.
  - Any required and provided interfaces and shared resources used by the software component. A component can add demand for system-level resources that might not be accounted for.
- The component's **build configuration** (how it was built so that it can be duplicated in the future) and sources
- Any **existing bugs and their workarounds**

SBOMs Scope Today

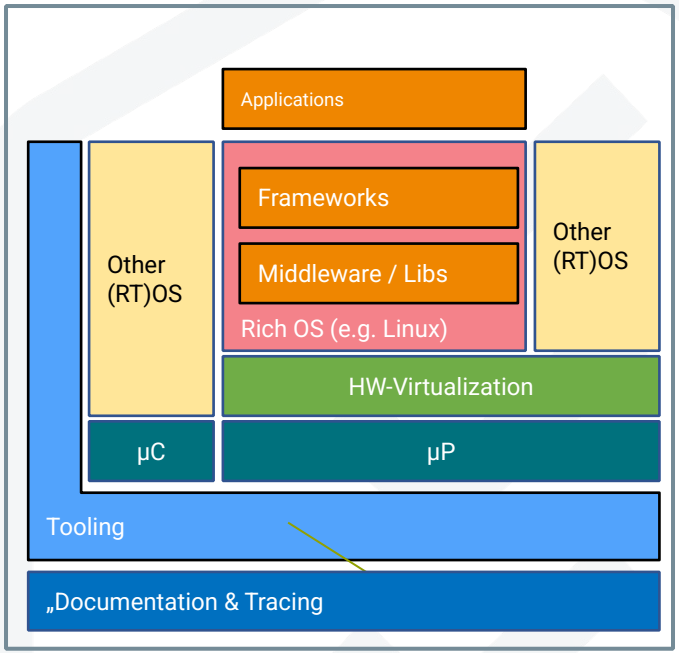
- **Documentation** for application manual for the component
  - The **intended use** of the software component
  - **Instructions** on how to **integrate** the software component correctly and **invoke it properly**
- **Requirements** for the software component
  - Coverage for nominal operating conditions and behavior in the case of failure
  - For highly safety critical requirements, test coverage should be in accordance with what the specification expects (e.g., Modified Condition/Decision Coverage (MC/DC) level code coverage)
  - Any safety requirements that might be violated if the included software performs incorrectly. This is specifically looking for failures in the included software that can cause the safety function to perform incorrectly. (This is referred to as a cascading failure.)
  - What the software might do under anomalous operating conditions (e.g., low memory or low available CPU)
- **Evidence**
  - This should include the results of any testing to demonstrate requirements coverage

Source: https://www.linux.com/featured/sboms-supporting-safety-critical-software/

A system is designed of many different parts

- Requirement tools
- Documentation frameworks
- Traceability tools
- Testing frameworks
- Build tooling
- Compiler
- Hypervisor
- (RT)OS
- Container
- Middleware
- Frameworks
- IDEs
- …

Projects
&
Foundations



ELISA
Enabling Linux in
Safety Applications

ELISA Project



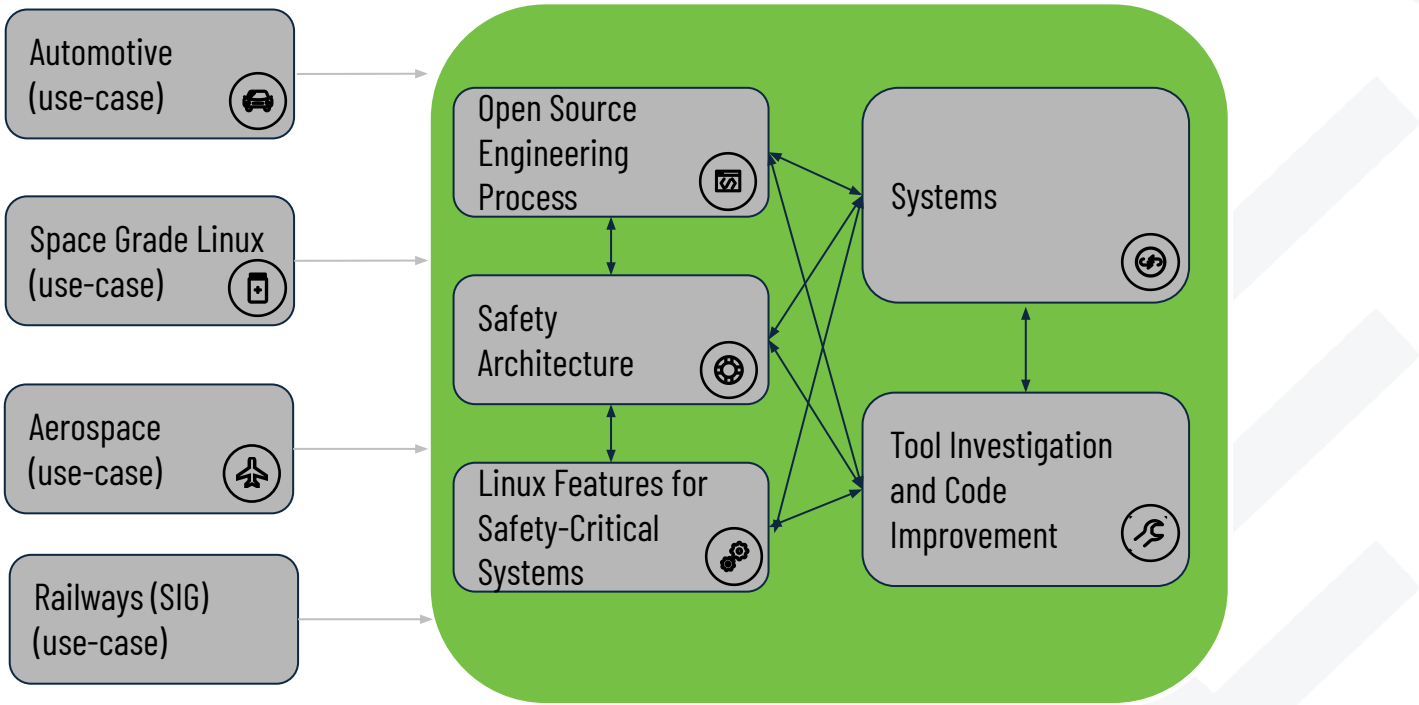
- Enabling **Safety-critical applications** with **Linux** (beyond Security)
- Increase **dependability & reliability** for whole Linux ecosystem
- **Various use cases**: Aerospace, Automotive, Trains, Medical & Industrial
- Supported by major **industrial grade Linux distributors** known for mission critical operation and various industries representatives
- Close community collaboration with **Xen, Zephyr, SPDX, Yocto & AGL** projects
- **Reproducible system** creation from specification to testing
- SW **elements**, engineering **processes**, development **tools**

ELISA : Architecture | Processes | Features | Tools | Systems

ELISA
Enabling Linux in Safety Applications

Work in Progress - License: CC-BY-4.0

Current Verticals & Horizontal Working Groups



ELISA
Enabling Linux in Safety Applications

Work in Progress - License: CC-BY-4.0



ELISA
Enabling Linux in
Safety Applications

Work in Progress - License: C

Linux in Safety Critical Systems

“Assessing whether a system is safe, requires understanding the system sufficiently.”

- Understand Linux within that system context and how Linux is used in that system.
- Select Linux components and features that can be evaluated for safety.
- Identify gaps that exist where more work is needed to evaluate safety sufficiently.

Work in Progress - License: CC-BY-4.0

Risk Management in Systems with Linux

- Software
  - Functions are increasingly defined by software
  - Managing interaction between sensors, actuators, humans & environment
  - Managing trained AI/ML models that assist in the safe & efficient operation
- Training Data Sets
  - Data used to train, test & validate the AI/ML models in used the system
- Communication to Remote Services
  - External environment awareness for navigation support
  - Updates to the software, firmware & models
- Hardware
  - Traditional devices, with increasing CPUs, MCUs, GPUs and FPGAs incorporated

Work in Progress - License: CC-BY-4.0

"Safe Systems with Linux“ MC @ LPC 2026

ELISA
Enabling Linux in Safety Applications

Addressing code

- Program Verification
- Practical Qualification efforts

  (common pitfalls & errors)

And things around it (like tools)

- Testable expectations
- Linux Kernel Requirements
- Safety SBOM for Linux



videos & slides at: https://lpc.events/event/20/sessions/254/#20261005

LINUX PLUMBERS CONFERENCE PRAGUE, CZECHIA / OCT. 5-7, 2026

Separating Fact From Interpretation

How an approach meets functional safety needs is rarely **public**:

- **Assessment:** the assessor issues a certificate and a report.
- **Private:** the report and any evidence stay confidential.
- **Published:** only code under legal obligation (e.g. GPLv2).
- **Result:** comparing approaches is often practically impossible.

The facts: safety concepts mapped onto Linux.

Frequent Misconceptions

No single "Linux": every version, build and hardware mix differs.

Kernel process: helps, but isn't aligned with safety standards.

Functional Correctness: strong evaluations help, but don't imply safety.

Functional Availability: safety often doesn't require it.

Stress testing: useful, but can't cover all plausible scenarios.

Monitoring: dependencies, observability and common cause failure.

Upstreaming: current practices can't prevent regressions in mechanisms.

Misconceptions on Configuration

Config flags: useful, but alone don't make a kernel safe.

PREEMPT_RT: useful, but alone doesn't make a kernel safe.

Security features: partial overlap with safety, but different goal.

Resources Management: partial overlap with safety, but different goal.

Like the kernel process, its configuration knobs alone are not sufficient for safety.

Safety Is Targeted

Safety mechanisms are **expensive** and **focused on use cases**.

Expensive
Safety does not come for free

- Every mechanism takes **effort**.
- Expecting **all drivers** to become safe **without effort** is not reasonable.

Focused
Driven by the use case

- Mechanisms target **specific use cases**.
- Making every driver safe is **usually not necessary**.

Make safe what the use case needs, not every driver.

source:https://lpc.events/event/20/contributions/2492/attachments/2006/4617/Common%20pitfalls%20and%20errors%20when%20using%20Linux%20in%20Safety%20Applications_.pdf

Work in Progress - License: CC-BY-4.0



ELISA
Enabling Linux in
Safety Applications

Call for Collaboration

Aerospace · Automotive · Linux Features · OS Engineering Process · Railways · Safety Architecture · Systems · Tools · Lighthouse · Space Grade

JOIN THE COMMUNITY

ELISA
Enabling Linux in Safety Applications

Our infrastructure and tools are open by default, so jump in and introduce yourself, ask questions and share ideas. Please consider this your invitation to participate.



Subscribe to Mailing Lists



Join Community Meetings



Contribute to Tools and Docs on GitHub



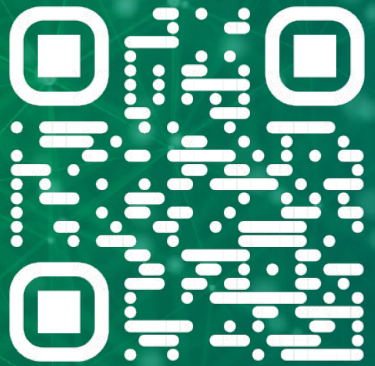
Participate in Working Groups



Attend Events

ELISA
Enabling Linux in Safety Applications

Thank you!



https://elisa.tech

Aerospace · Automotive · Linux Features · OS Engineering Process · Railways · Safety Architecture · Systems · Tools · Lighthouse · Space Grade